

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..31

SERVER-SIDE HTML

 Copy as Markdown

Server-side HTML

Server-rendered pages generate their HTML on the server for each request, with full access to your database through the `Session`. Choose this for content pages, dashboards, and landing pages. For client-rendered apps, see [Single-page apps](#) instead.

Serverpod's built-in mechanism for this is `WidgetRoute` with [Mustache](#) templates. Beyond it, any route can return HTML that you produce with a tool of your choice. This page covers both, using [Jaspr](#), which brings a Flutter-like component model to the web, as the worked example of the second approach.

Creating a WidgetRoute

Create custom routes by extending the `WidgetRoute` class and implementing the `build` method:

```
class MyRoute extends WidgetRoute {
  @override
  Future<WebWidget?> build(Session session, Request request) async {
    return MyPageWidget(title: 'Home page');
  }
}

// Register the route
pod.webServer.addRoute(MyRoute(), '/my/page/address');
```

The `build` method returns the widget that renders the page. In this example, that widget is `MyPageWidget`, which the [next section](#) defines. Return `null` to respond with 404, for example when the requested content does not exist.

Like other routes, a `WidgetRoute` answers GET requests by default. Pass `methods:` to handle submissions too, e.g. `super(methods: {Method.get, Method.post})`. Rendered responses with `Cache-Control: no-cache, private`, so browsers and CDNs revalidate them on every request. The exception is `RedirectWidget`, whose 303 response carries no cache header.



Ask AI

Creating a TemplateWidget

A `TemplateWidget` renders a Mustache template. It is a subclass of `WebWidget`, which is what allows `MyRoute`'s `build` method above to return one. A template widget consists of a Dart class and a corresponding HTML template file. Create one by extending `TemplateWidget`:

```
class MyPageWidget extends TemplateWidget {
  MyPageWidget({required String title}) : super(name: 'my_page') {
    values = {
      'title': title,
    };
  }
}
```

Place the corresponding HTML template in the `web/templates` directory. The HTML file uses the `Mustache` template language:

```
<!DOCTYPE html>
<html>
<head>
  <title>{{title}}</title>
</head>
<body>
  <h1>{{title}}</h1>
  <p>Welcome to my page!</p>
</body>
</html>
```

Template values are converted to `String` objects before being passed to the template. Mustache escapes HTML in `{{value}}` tags, so to embed HTML produced by a nested widget, use the unescaped `{{{value}}}` form.

How templates load

Templates are loaded from `web/templates` when the server starts. Constructing a `TemplateWidget` whose template file is missing throws a `StateError` naming the widget class, e.g. `Template my_page.html missing for MyPageWidget`, and the request fails with a 500. While `serverpod start` runs, templates reload on save and the browser refreshes automatically. With a plain `dart run bin/main.dart`, restart the server to pick up template changes.

Cache-busted asset URLs

A template can refer to a stylesheet or an image by its `cache-busted` URL with the `{{{@/path}}}` tag, so the browser picks up a new version of the file as soon as it changes. Write the asset's served path after the `@`:

```
web/templates/my_page.html
```

```
<link rel="stylesheet" href="{{@/static/css/style.css}}">

```

Pass the same `CacheBustingConfig` as the `StaticRoute` serving the directory to the route that renders the template:

```
class MyRoute extends WidgetRoute {
  MyRoute() : super(cacheBustingConfig: cacheBustingConfig);

  @override
  Future<WebWidget?> build(Session session, Request request) async {
    return MyPageWidget(title: 'Home page');
  }
}
```

To build the versioned URL in Dart instead and pass it in as a template value, use `assetPath()`.

Built-in widgets

Serverpod provides several built-in widgets for common use cases:

- `ListWidget` - Concatenates multiple widgets into a single response

```
return ListWidget(widgets: [
  HeaderWidget(),
  ContentWidget(),
  FooterWidget(),
]);
```

- `JsonWidget` - Renders JSON documents from serializable data structures

```
return JsonWidget(object: {'status': 'success', 'data': myData});
```

- `RedirectWidget` - Redirects the browser to another URL with a `303 See Other` response

```
return RedirectWidget(url: '/new/location');
```

Database access and logging

The web server passes a `Session` object to the `WidgetRoute` class' `build` method. This gives you access to all the features you typically get from a standard method call to an endpoint. Use the database, logging, or caching the same way you would in a method call:

```

class DataRoute extends WidgetRoute {
  @override
  Future<WebWidget?> build(Session session, Request request) async {
    // Access the database
    final users = await User.db.find(session);

    // Logging
    session.log('Rendering user list page');

    return UserListWidget(users: users);
  }
}

```

Rendering with Jaspr

Jaspr lets you build server-rendered pages from components, with a **build** method and composition model much like Flutter's. It works with the web server through a plain **Route**: your route renders a component to HTML and returns it as the response.

Add the dependency to your server package (the example on this page uses jaspr 0.23):

```
$ dart pub add jaspr
```

Then define a component and a route that renders it:

```

import 'dart:convert';

import 'package:jaspr/dom.dart' as jaspr;
import 'package:jaspr/server.dart' as jaspr;
import 'package:serverpod/serverpod.dart';

class HelloPage extends jaspr.StatelessComponent {
  const HelloPage({required this.name, super.key});

  final String name;

  @override
  jaspr.Component build(jaspr.BuildContext context) {
    return jaspr.div([
      jaspr.h1([jaspr.Component.text('Hello from Jaspr!')]),
      jaspr.p([jaspr.Component.text('Rendered server-side for $name.')] ),
    ]);
  }
}

class JasprHelloRoute extends Route {
  @override
  Future<Result> handleCall(Session session, Request request) async {
    final result = await jaspr.renderComponent(HelloPage(name: 'Serverpod'));
    return Response.ok(

```

```
body: Body.fromString(  
  utf8.decode(result.body),  
  mimeType: MediaType.html,  
),  
);  
}  
}
```

Register the route as usual, and initialize Jaspr once in your `run()` function before the server starts:

```
jaspr.Jaspr.initializeApp();  
  
pod.webServer.addRoute(JasprHelloRoute(), '/jaspr');
```

WARNING

The `initializeApp()` call is required. Without it, the first render kills the whole server process instead of failing the single request.

Two details make this example work. Both `package:jaspr/server.dart` and `package:jaspr/dom.dart` are imported under one alias, because `server.dart` exports names that collide with the web server's `Request` and `Response`, and the HTML components (`div`, `h1`, `p`) live in `dom.dart`. And `renderComponent` returns a result record whose `body` is bytes, which `utf8.decode` turns into the HTML string.

Next steps

- Use [custom routes](#) for REST APIs and custom request handling
- Serve [static files](#) for CSS, JavaScript, and images
- Add [middleware](#) for cross-cutting concerns like logging and error handling